
Practical Homework 1

Audio Enhancement & Image-Based Fracture Detection

Course: Signals and Systems

Prepared by

Soheil Sayah Varg • Masiha Bagheri Tanha

Submission format: Jupyter Notebook (.ipynb) + exported PDF

Due date: 8 Azar (29 November), 23:55

Total credit: 100 points (Problem 1: 50 – Problem 2: 50)

A hands-on assignment on real-world signal restoration and image analysis.

July 6, 2026

Contents

1 General Instructions & Guidelines 1

2 Problem 1 — Call-Center Audio Restoration & Pre-Processing 1

2.1 Context 1

2.2 Observed Artifacts to Address 1

2.3 Required Tasks & Grading Rubric (50 points) 2

2.4 Suggested Approach & Hints 2

3 Problem 2 — Automated Wellbore Fracture Segmentation 3

3.1 Context 3

3.2 Structural Definitions 3

3.3 Required Tasks & Grading Rubric (50 points) 4

3.4 Suggested Approach & Hints 4

4 Submission Checklist 4

1 General Instructions & Guidelines

- **Environment.** All code must be implemented in **Python 3** inside a Jupyter Notebook.
- **Permitted libraries.** You may use standard scientific libraries: `numpy`, `scipy`, `matplotlib`, `librosa`, and `opencv-python (cv2)`.
- **Documentation.** Your code must be modular and well commented. Accompany it with Markdown cells that explain your design choices, algorithmic steps, and a short analysis of the results.
- **Generative-AI policy.** You may use AI tools (e.g. ChatGPT, Gemini) for *debugging or brainstorming*. However, you **must** explicitly document your prompts and explain the underlying signal-processing theory behind any generated code. Plagiarism or pasting code you do not understand will result in **zero credit**.
- **Deliverable.** Submit both the executed `.ipynb` and its exported PDF, with all figures and audio/image outputs rendered.

A note on the provided files

The audio files are supplied as `2.mp3`, `3.mp3`, `4.mp3` and the borehole images as `Well11.jpg` and `well12.png` (with additional examples in the *More Well images* folder). Load them by their actual file names — e.g. `librosa.load("2.mp3")` and `cv2.imread("Well11.jpg")`. Resample the audio to a consistent rate if you wish; the originals are 44.1 kHz.

2 Problem 1 — Call-Center Audio Restoration & Pre-Processing

2.1 Context

In real-world telecommunication systems (VoIP, public switched telephone networks), voice signals are frequently degraded by the acoustic environment, channel limitations, and network artifacts. You are given three audio recordings of call-center conversations. Your goal is to design an **audio-enhancement pipeline** that isolates, cleans, and normalizes the human speech.

2.2 Observed Artifacts to Address

1. **Bandwidth limitations.** Low-pass / band-pass effects from standard telephone channel constraints ($\approx 300\text{ Hz} - 3.4\text{ kHz}$).
2. **Acoustic noise.** Stationary background noise (hiss / hum) and non-stationary transient sounds.
3. **Echo interference.** Delayed and attenuated reflections of the speech signal.
4. **Voice activity & silences.** Large blocks of dead-air / silence, or background hold music (*Aahang-e Entezaar*).
5. **Amplitude fluctuations.** Inconsistent volume levels between speakers or over time.

What the three recordings look like (quick characterization)

A fast exploratory pass reveals distinct challenges per file, so treat each separately:

- **2.mp3 & 4.mp3:** energy essentially confined below $\sim 3\text{ kHz}$ (classic telephone band) with *large silent stretches* ($\approx 60\%$ of the duration) — ideal for demonstrating band-pass filtering and Voice Activity Detection.
- **3.mp3:** loud and near-continuous (very little silence) with broadband content — a good target for the hold-music / echo strategy and for volume normalization.

Confirm these observations yourself with waveforms and spectrograms; do not simply quote them.

2.3 Required Tasks & Grading Rubric (50 points)

Milestone / Component	Requirements	Points
Exploratory Data Analysis (EDA)	Plot time-domain waveforms and spectrograms (<code>stft</code>), identifying where noise, echo, and hold music occur.	10
Volume Normalization & VAD	Implement a simple RMS- or energy-based threshold to strip out long silences (Voice Activity Detection).	10
Noise & Bandwidth Handling	Apply standard band-pass filters (e.g. Butterworth) to isolate speech frequencies, or implement a basic spectral-subtraction algorithm.	15
Echo / Hold-Music Strategy	Attempt an echo-cancellation heuristic (e.g. autocorrelation-based delay detection) or a frequency notch-filter for the music. <i>Graded on effort and logical approach rather than perfect removal.</i>	10
Comparative Analysis & Report	A text summary evaluating which artifacts were easiest / hardest to remove, comparing before/after audio-quality metrics.	5
Total		50

2.4 Suggested Approach & Hints

- **EDA.** Use `librosa.stft` and `librosa.display.specshow` to see the band limitation (energy cut-off), noise floor, and repetitive/tonal hold music.

- **VAD.** Compute short-time RMS/energy (`librosa.feature.rms`), set a threshold relative to the maximum (or noise floor), and mask/remove frames below it. Add hysteresis or a minimum-silence duration to avoid chopping words.
- **Band-pass.** Design a Butterworth filter with `scipy.signal.butter` / `sosfiltfilt` over roughly 300 Hz–3.4 kHz.
- **Spectral subtraction.** Estimate the noise magnitude spectrum from a silent segment and subtract it from every frame before inverse STFT.
- **Echo.** The autocorrelation of the signal peaks at the echo delay; a comb/notch filter or a simple inverse filter can then attenuate it.
- **Metrics.** Report SNR estimates, RMS levels, or spectral flatness before vs. after each stage to justify your choices.

3 Problem 2 — Automated Wellbore Fracture Segmentation

3.1 Context

Borehole acoustic/optical imaging is a crucial tool in petroleum engineering for assessing the structural integrity of a well. The unwrapped image of the wellbore wall displays complex networks of natural and induced fractures. Your task is to build an **automated image-processing pipeline** that detects and isolates the *primary structural fractures* from the images `Well11.jpg` and `well12.png`.

3.2 Structural Definitions

- **Primary fractures.** Distinct, continuous curves that span horizontally across the *entire* unwrapped borehole view, maintaining a high relative pixel thickness (diameter). **These are what you must keep.**
- **Secondary fractures.** Thin, localized, or fragmented micro-cracks — these should be *filtered out*.

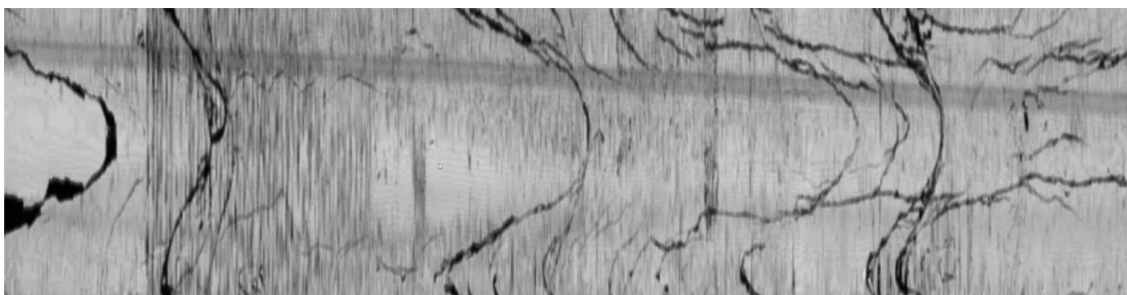


Figure 1: `Well11.jpg` — an unwrapped borehole scan. Note the thick, continuous primary fractures crossing the width, the many thin micro-cracks, and the vertical striping artifact.

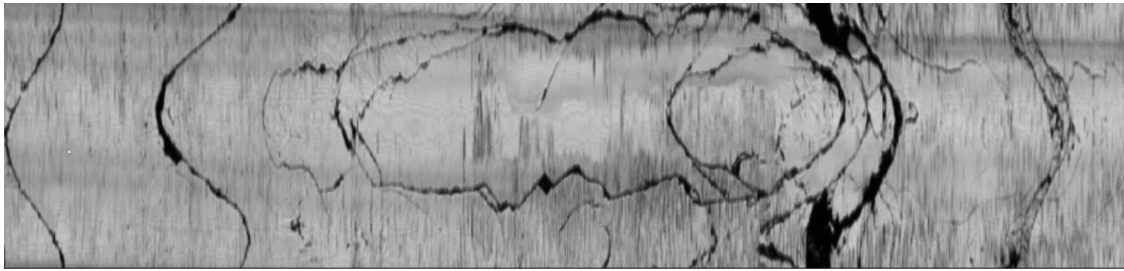


Figure 2: `well12.png` — a second scan with sinusoidal fracture traces (planar fractures appear as sinusoids in the unwrapped view) plus overlapping fine cracks to be suppressed.

3.3 Required Tasks & Grading Rubric (50 points)

Milestone / Component	Requirements	Points
Pre-processing & Filtering	Grayscale conversion, contrast adjustment (CLAHE), and blurring (Gaussian / median) to suppress fine noise.	10
Edge & Ridge Detection	Use Sobel, Canny, or Frangi vessel/ridge filters to highlight the curved structures.	15
Morphological Selection	Use morphological operations (dilation, erosion, opening/closing with specific structuring elements) to eliminate thin micro-cracks while preserving the thick lines.	15
Continuity Verification	An algorithm or heuristic (e.g. contour-length filtering, Connected-Components Analysis) confirming that the selected fractures cross the image horizontally.	10
Total		50

3.4 Suggested Approach & Hints

- **Pre-processing.** Convert to grayscale, apply `cv2.createCLAHE` for local contrast, then a median/Gaussian blur to knock down the vertical striping and speckle.
- **Ridge detection.** Fractures are dark elongated ridges; `skimage.filters.frangi` (or `meijering/sato`) responds strongly to curvilinear structures. Canny / Sobel are simpler alternatives.
- **Thickness selection.** A morphological *opening* with a structuring element wider than the micro-cracks removes thin lines while thick primary fractures survive; follow with a *closing* to reconnect gaps.
- **Continuity.** Label connected components (`cv2.connectedComponentsWithStats` or `skimage.measure.label`) then keep only components whose horizontal extent (bounding-box width) approaches the image width and whose length exceeds a threshold.
- **Validation.** The *More Well images* folder contains extra scans you can use to check that your pipeline generalizes beyond the two graded images.

4 Submission Checklist

- ☐ A single Jupyter Notebook (`.ipynb`) containing both problems, run top-to-bottom without errors.

- ☐ An exported **PDF** of the executed notebook (figures and results visible).
- ☐ Markdown explanations of the theory and design choices for every stage.
- ☐ Before/after comparisons (audio spectrograms, processed images) for each problem.
- ☐ Documented AI prompts, if any tools were used.
- ☐ Submitted by **1 Mordad (July 23), 23:59**.

Good luck — focus on sound signal-processing reasoning, not just working code.